#4 We compute the distance between current node's point and target location.
#5 We use the `cmpShops` method to decide which of the shops better satisfies our needs.
#6/#7 Check if the target point is on the left branch of the split. If it is, the left branch is the closest to the target point, otherwise it is the furthest.
#8 We certainly need to traverse the closest branch in search of the nearest neighbor. It is important to do so first and update the mementos for NN's distance, to improve pruning.
#9 Using one of the helper functions defined in listing 9.2, we compute the distance between the split line passing through current node and the target point. If this distance is closer than the distance to current nearest neighbor, then the furthest branch might contain points closer than current nearest neighbor (see figure 9.21).
#10 Traverse furthest branch, and update current values for NN and its distance.
#11 Return the closest point found so far.

When we pass a compare function to `fNN`, instead, this flexibility is not going to be possible, and we will only examine solutions where all items are shipped by the same shop.

If that's fine, if we have guarantees that we can always find such a shop (or if we handle separately the case where we fail to find one), the "compare function" mechanism has the advantage of requiring less extra memory and being overall faster.